Part 5 – Rohan: Error analysis and explainability (Part E)

1. What my part is

Part E asks two questions about our best model: where does it go wrong, and what does it look at when it is right? The model is MobileNetV2, fully fine-tuned, seed 0 (`models/mobilenet_v2_full_frac1_seed0.pth`), scored on the 4,050 test tiles that were never used for training or for choosing anything. The main files are `src/evaluate.py` (predictions, confusion matrix, per-class report, list of mistakes), `src/gradcam.py` (Grad-CAM written by hand) and `notebooks/04_evaluation_and_explainability.ipynb`, which all three appear in my log. The other Part E scripts are the Grad-CAM runs over all classes, the confidence-drop test, its control, the t-SNE plot, the two error-picture scripts and `src/train_part_e.py`, the seed-4 scratch CNN runs. The headline result is 3,981 of 4,050 right (98.30% accuracy on the CPU re-scoring, 98.20% macro-F1) and 69 mistakes. The most useful finding is that the mistakes are concentrated in the crop and vegetation classes. The least useful one, which we must say honestly, is that Grad-CAM on this model does not point at anything more important than the rest of the image. The t-SNE plot is a picture of the features, not a measurement.

2. Definitions

Test split. The 4,050 images in `data/splits.csv` marked `test`. They are used once to score a finished model and never to make a decision.

Checkpoint (weights file). The saved numbers of a trained model (`.pth` file). Loading it gives back the trained model without training again.

Prediction. The class the model says an image belongs to. It is the class with the highest score, found with `argmax`.

Confusion matrix. A 10 x 10 table. Each row is the real class, each column is the predicted class. The diagonal holds the correct answers; every off-diagonal cell counts one kind of mistake.

True positive / false positive / false negative. For one class: a true positive is an image of the class called that class; a false positive is an image of another class called this class; a false negative is an image of this class called another class.

Precision. Of all images the model called class X, the share that really were X. Low precision means X is called too often.

Recall. Of all real images of class X, the share the model found. Low recall means X is missed too often.

F1 score. The harmonic mean of precision and recall, `2 x P x R / (P + R)`. It is high only when both are high.

Accuracy. Correct predictions divided by all predictions. It is dominated by the bigger classes.

Macro-F1. The F1 of each class, then the plain average of the 10 values. Every class counts the same, whatever its size.

Support. The number of real test images of a class (for example 450 AnnualCrop, 300 Pasture).

Classification report. A table with precision, recall, F1 and support for each class, plus the accuracy and the macro and weighted averages. Saved as `results/mobilenet_v2_full_frac1_seed0_classification_report.csv`.

Most-confused pair. A pair (actual class, predicted class) with many mistakes, for example 7 AnnualCrop images called PermanentCrop.

Misclassified list. The file `results/misclassified_images.csv`: one row per wrong test image with its path, the real and the predicted class.

Activation. The numbers a layer produces for one image. A "feature map" is one channel of those numbers, a small grid. For our 64 x 64 tile, the last MobileNetV2 block gives 1,280 maps of 2 x 2.

Gradient. How much the output score would change if one number in a layer were changed a little. A positive gradient means "more of this feature increases the score".

Forward hook. A function PyTorch calls automatically when data passes forward through a layer. Ours copies the activations out.

Backward hook. A function PyTorch calls when gradients pass backward through a layer. Ours copies the gradients out.

Backward pass (backpropagation). Computing the gradient of a score with respect to every weight and activation, starting from the output and going backward through the network. `score.backward()` does this.

Grad-CAM. Gradient-weighted Class Activation Mapping. It builds a coarse heat map showing which regions of the last convolution layer pushed the chosen class score up.

Global average pooling. Replacing each feature map by its mean. It turns 1,280 maps of 2 x 2 into 1,280 single numbers. Our classifier and our t-SNE features both use it.

ReLU. The function `max(0, x)`. Negative values become 0. In Grad-CAM it keeps only the regions that push the score up.

Heat map. A picture of numbers, here shown with the colour map "jet" (blue for low, red for high).

Overlay. The heat map drawn semi-transparently on top of the original image.

Bilinear interpolation. Smooth resizing of a small grid to a big one by blending neighbouring values. We use it to stretch the 2 x 2 map to 64 x 64.

Saliency. A general name for maps that show which input pixels matter for a prediction. Grad-CAM is one kind of saliency map.

Softmax. A function that turns the 10 raw scores into probabilities that add up to 1.

Confidence. Here, the softmax probability of the class the model chose (or, in the confidence-drop test, of the true class).

Control experiment. A second test run in the same way as the first, but with a different choice, so that we can tell whether the first result means anything. Ours compares the hottest patch with the coldest patch and a random patch.

Confidence drop. How much the confidence falls after part of the image is hidden. A big fall suggests that the hidden part mattered.

Embedding (feature vector). The list of numbers that a network produces for one image before the final classifier. Here, 1,280 numbers per image.

t-SNE. t-distributed Stochastic Neighbour Embedding. It places high-dimensional points in 2 dimensions so that points that were close stay close. It is for looking, not for measuring.

Perplexity. A t-SNE setting that says roughly how many neighbours each point should consider. We use 30.

PCA initialisation. Starting the t-SNE layout from the main directions of the data (principal component analysis) instead of random positions. It makes the result more repeatable.

Random seed. A number that fixes the random choices, so the same run gives the same result. Our seed for the t-SNE sample is 42.

Stratified split and data fraction. A split that keeps each class in the same share. A fraction of 0.05 means 5% of the training images, chosen per class.

Seed-4 runs. Extra scratch CNN training runs with seed 4 at 5%, 10% and 25% of the training images, stored in `results/metrics.csv`.

Normalisation (ImageNet mean and std). Each colour channel is shifted and scaled with fixed numbers (mean 0.485, 0.456, 0.406; std 0.229, 0.224, 0.225). The value 0 after normalisation is therefore the ImageNet average colour, not the average colour of EuroSAT.

`model.eval()` and `torch.no_grad()`. `eval()` switches off training-only behaviour (dropout, batch-norm updates). `no_grad()` stops PyTorch from storing gradients when we only need predictions.

Quantile and top-k. `quantile` gives the value below which a share of the numbers fall. `topk` gives the positions of the largest values. We use them to pick the hottest and coldest pixels.

3. Code, file by file

Each block is copied from the file. Line numbers refer to that file. Where a block is a notebook cell, the number is the block's own line count.

3.1 `src/evaluate.py`

Header and imports.

```
"""evaluate.py - how good is the model, and where does it fail?

Owner: Rohan
Part: E (Evaluation + Grad-CAM + t-SNE)
"""

import argparse

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
)
from torchvision.datasets import EuroSAT

from src.dataset import get_loader
from src.train import build_model
from src.utils import DATA_DIR, MODELS_DIR, RESULTS_DIR, FIGURES_DIR, get_device
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
2-6	Docstring: says what the file is for and who owns it	Documentation	Lets a teammate know the file without reading all of it
8	Imports `argparse`	Standard library	Lets the file be run from the command line with `--model`, `--fraction`, `--seed`

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
10-13	Plotting, numbers, data tables, PyTorch	Libraries	Used for the plot, the arrays, the CSV files and the model
14-19	Four metrics from scikit-learn	scikit-learn	Using the library avoids writing our own F1 or confusion-matrix code, which is easy to get wrong. Could have been written by hand with counts
20	Imports the EuroSAT class list	torchvision	Gives the 10 class names in the right order (the same order as the labels 0-9)
22-24	Our own helpers: loader, model builder, folder paths, device	Project code	Every script uses the same paths and the same test loader, so results match

The function `evaluate_model`, first part: builds the model, loads the saved weights, makes the test loader.

```
def evaluate_model(model_name, fraction=1.0, seed=0, batch_size=64):

    device = get_device()

    # Build model architecture
    model, _ = build_model(model_name)

    # Find checkpoint
    checkpoint_path = MODELS_DIR / (
        f"{model_name}_frac{fraction:g}_seed{seed}.pth"
    )

    if not checkpoint_path.exists():
        raise FileNotFoundError(f"Checkpoint not found: {checkpoint_path}")

    # Load trained weights
    state = torch.load(
        checkpoint_path,
        map_location=device,
        weights_only=True,
    )
    model.load_state_dict(state)
    model.to(device)
    model.eval()

    print("Model:", model_name)
    print("Checkpoint:", checkpoint_path)
    print("Device:", device)

    # Load test dataset
    test_loader = get_loader(
        "test",
        fraction=1.0,
        seed=0,
        augment=False,
        batch_size=batch_size,
        num_workers=0,
    )
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
27	Function takes model name, fraction, seed, batch size	Function with defaults	Defaults are the final model; `fraction` and `seed` let the same code score the seed-4 runs
29	Chooses GPU if there is one, else CPU	Device	Our final run was on the CPU; the same code works on either
32	Builds an empty model of the right shape	Model architecture	Weights can only be loaded into a model of the same shape
35-37	Builds the file name, e.g. `mobilenet_v2_full_frac1_seed0.pth`	File naming	The name carries the run settings so files do not get mixed up

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
39-40	Stops with a clear error if the weights are missing	Defensive check	Better an error message than a silent wrong result
43-47	Loads the weights, `weights_only=True`	`torch.load`	`weights_only` only reads numbers, so an untrusted file cannot run code. Could load the whole object instead, which is less safe
48-50	Puts the weights in the model, moves it to the device, switches to evaluation mode	`load_state_dict`, `eval()`	`eval()` turns off dropout and stops batch-norm statistics from changing
52-54	Prints what was loaded	Logging	Lets us see in the output which checkpoint and device were used
57-64	Builds the test loader, no augmentation, no shuffling	DataLoader	Augmentation is for training only. The test order must stay fixed so the saved mistakes list matches

```
actual_labels = []
predicted_labels = []

# Predict all test images
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)

        outputs = model(images)
        predictions = outputs.argmax(dim=1)

        actual_labels.extend(labels.tolist())
        predicted_labels.extend(predictions.cpu().tolist())

y_true = np.array(actual_labels)
y_pred = np.array(predicted_labels)

# Save the paths of incorrectly predicted test images
test_df = test_loader.dataset.df.reset_index(drop=True)

class_names = EuroSAT(
    root=str(DATA_DIR),
    download=False,
).classes
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
70-72	Turns off gradient storing, sends each batch to the device	`torch.no_grad()`	Saves memory and time; we only need the outputs
74-75	Runs the model and takes the highest score per image	Prediction, `argmax`	The predicted class is the one with the biggest score
77-78	Stores the true and predicted labels as plain lists	Bookkeeping	Lists are easy to turn into arrays for scikit-learn
80-81	Converts the lists to NumPy arrays	NumPy	Arrays allow comparisons like `y_true != y_pred` in one step
84	Keeps the table of test rows, so each mistake can be traced to a file	DataFrame	Needed to write the paths of the wrong images
86-89	Reads the 10 class names from torchvision	Class names	Turns numbers 0-9 into names for the tables

```
mistakes = test_df.iloc[np.where(y_true != y_pred)[0]].copy()

mistakes["actual_class"] = [
```

```
    class_names[label] for label in y_true[y_true != y_pred]
]

mistakes["predicted_class"] = [
    class_names[label] for label in y_pred[y_true != y_pred]
]

mistakes["actual_label"] = y_true[y_true != y_pred]
mistakes["predicted_label"] = y_pred[y_true != y_pred]

mistakes_path = RESULTS_DIR / "misclassified_images.csv"
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

mistakes.to_csv(mistakes_path, index=False)

print("\nMisclassified images saved:", mistakes_path)
print("Total misclassified images:", len(mistakes))

# EuroSAT class names
class_names = EuroSAT(
    root=str(DATA_DIR),
    download=False,
).classes
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
91	Selects the rows where the answer is wrong	Boolean indexing	One line gives the 69 mistakes
93-98	Adds the real and the predicted class names to those rows	Mapping numbers to names	A person can read "AnnualCrop -> PermanentCrop" but not "0 -> 6"
101-102	Adds the numeric labels too	Bookkeeping	Scripts that come later (the Grad-CAM runs) use the numbers
104-107	Saves the list to `results/misclassified_images.csv`	CSV output	Other scripts read this file to pick examples by a fixed rule
109-110	Prints how many mistakes there are	Logging	Quick check: 69

Note: lines 86-89 and 113-116 load the same class names twice. This does no harm, but it is redundant.

```
# Calculate metrics
accuracy = accuracy_score(y_true, y_pred)
macro_f1 = f1_score(
    y_true,
    y_pred,
    average="macro",
    zero_division=0,
)

cm = confusion_matrix(
    y_true,
    y_pred,
    labels=list(range(len(class_names))),
)

report = classification_report(
    y_true,
    y_pred,
    labels=list(range(len(class_names))),
    target_names=class_names,
    output_dict=True,
    zero_division=0,
)

# Create output folders
RESULTS_DIR.mkdir(parents=True, exist_ok=True)
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

name = f"{model_name}_frac{fraction:g}_seed{seed}"

# Save confusion matrix
```

```
    cm_path = RESULTS_DIR / f"{name}_confusion_matrix.csv"

    pd.DataFrame(
        cm,
        index=class_names,
        columns=class_names,
    ).to_csv(cm_path)

    # Save classification report
    report_path = RESULTS_DIR / f"{name}_classification_report.csv"

    pd.DataFrame(report).transpose().to_csv(report_path)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
119	Accuracy: correct divided by all	Accuracy	The first number everyone asks for
120-125	Macro-F1 with `average="macro"`	Macro-F1	Gives each class equal weight. `zero_division=0` returns 0 instead of an error for a class that is never predicted
127-131	Confusion matrix, with the labels 0-9 forced in order	`confusion_matrix`	Forcing the labels keeps the rows and columns in the same order even if a class is missing
133-140	Classification report as a dictionary	`classification_report`	`output_dict=True` lets us save it as a table
143-144	Makes the output folders if missing	`mkdir`	The script runs on a new laptop without setup
149-155	Saves the confusion matrix as CSV with class names	CSV	The notebook reads this file
158-160	Saves the per-class report as CSV	CSV	Same reason; the file has precision, recall, f1-score and support

```
    # Plot confusion matrix
    fig, ax = plt.subplots(figsize=(9, 8))

    image = ax.imshow(cm, cmap="Blues")
    fig.colorbar(image, ax=ax)

    ax.set_xticks(range(len(class_names)))
    ax.set_yticks(range(len(class_names)))

    ax.set_xticklabels(class_names, rotation=45, ha="right")
    ax.set_yticklabels(class_names)

    ax.set_xlabel("Predicted class")
    ax.set_ylabel("Actual class")
    ax.set_title(f"Confusion Matrix - {model_name}")

    # Write numbers inside matrix cells
    for i in range(len(class_names)):
        for j in range(len(class_names)):
            ax.text(j, i, cm[i, j], ha="center", va="center")

    fig.tight_layout()

    plot_path = FIGURES_DIR / f"{name}_confusion_matrix.png"

    fig.savefig(plot_path, dpi=100)
    plt.close(fig)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
163-166	Creates a figure and a blue colour map of the matrix	matplotlib	Darker blue means more images in that cell
168-172	Sets tick labels and rotates the class names	Plot layout	Long class names need the rotation to be readable

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
174-176	Axis labels and title	Plot	"Predicted class" on the x-axis, "Actual class" on the y-axis (the reverse of many textbooks, so read the axes first)
179-181	Writes each count inside its cell	Annotation	Numbers are needed for the viva; colour alone is not enough
185-188	Saves at 100 dpi and closes the figure	Save	Closing frees memory when many plots are made

```
    # Display results
    print("\n========== EVALUATION RESULTS ==========")
    print("Test images:", len(y_true))
    print(f"Test accuracy: {accuracy:.4f}")
    print(f"Macro-F1:     {macro_f1:.4f}")

    print("\nPer-class report:")
    print(
        classification_report(
            y_true,
            y_pred,
            target_names=class_names,
            zero_division=0,
        )
    )

    print("Confusion matrix CSV:", cm_path)
    print("Classification report CSV:", report_path)
    print("Confusion matrix plot:", plot_path)

    return y_true, y_pred, cm, report
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
191-194	Prints test size, accuracy, macro-F1	Logging	The three numbers to remember
196-204	Prints the per-class table again	Logging	The same table as the CSV, for a quick look in the terminal
206-208	Prints where the files are	Logging	Lets us find the outputs
210	Returns the true labels, predictions, matrix and report	Return values	Lets another script or the notebook reuse them without running the model again

Main block (command line).

```
if __name__ == "__main__":

    parser = argparse.ArgumentParser()

    parser.add_argument("--model", default="mobilenet_v2_full")
    parser.add_argument("--fraction", type=float, default=1.0)
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument("--batch-size", type=int, default=64)

    args = parser.parse_args()

    evaluate_model(
        model_name=args.model,
        fraction=args.fraction,
        seed=args.seed,
        batch_size=args.batch_size,
    )
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
217-220	Reads the options from the command line, with defaults for the final model	`argparse`	Lets us score the seed-4 runs with the same file. Could hard-code the names instead

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
222-229	Calls the function with the options	Entry point	`if __name__ == "__main__"` means the code only runs when the file is run directly, not when it is imported

3.2 `src/gradcam.py`: Grad-CAM written by hand

This file is the core of the explainability part. The function takes one image (shape 1 x 3 x H x W), a layer, and returns a heat map (64 x 64, values 0 to 1) and the predicted class.

```
def grad_cam(model, image, layer):
    """Generate a Grad-CAM heatmap for one image.

    Args:
        model: trained classification model
        image: tensor shaped (1, 3, H, W)
        layer: target convolutional layer

    Returns:
        heatmap: tensor shaped (H, W), values between 0 and 1
        predicted_class: integer predicted class
    """

    model.eval()

    activations = None
    gradients = None

    # Save feature maps from the target layer
    def forward_hook(module, inputs, output):
        nonlocal activations
        activations = output

    # Save gradients flowing through the target layer
    def backward_hook(module, grad_input, grad_output):
        nonlocal gradients
        gradients = grad_output[0]

    forward_handle = layer.register_forward_hook(forward_hook)
    backward_handle = layer.register_full_backward_hook(backward_hook)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
11-22	Signature and docstring: input, layer, outputs	Function contract	States the shapes, so the caller knows what to pass
24	Sets the model to evaluation mode	`eval()`	Batch-norm and dropout must not change during the explanation
26-27	Two empty holders for activations and gradients	Closure, `nonlocal`	The hook functions write into these variables
30-32	Forward hook: copies the layer output	Forward hook, activation	This is the feature map A^k we need for the weighted sum
35-37	Backward hook: copies the gradient of the layer output	Backward hook, gradient	`grad_output[0]` is the gradient of the score with respect to the layer's output
39-40	Registers the two hooks on the chosen layer	`register_forward_hook`, `register_full_backward_hook`	The "full" backward hook is the version that gives the gradient of the output. Could use tensor hooks instead

```
    try:
        model.zero_grad()

        # Forward pass
```

```
        output = model(image)
        predicted_class = output.argmax(dim=1).item()

        # Backpropagate the predicted class score
        score = output[0, predicted_class]
        score.backward()

        # Average gradients across spatial dimensions
        weights = gradients.mean(dim=(2, 3), keepdim=True)

        # Weighted sum of feature maps
        cam = (weights * activations).sum(dim=1, keepdim=True)

        # Keep only positive importance
        cam = F.relu(cam)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
43	Clears old gradients	`zero_grad`	Without it, gradients from an earlier image would be added in
46-47	Runs the image and takes the predicted class	Forward pass, `argmax`	The explanation is for the class the model actually chose
50-51	Takes that class's score and backpropagates it	Backward pass	Gives the gradient of this one score for every activation
54	Weight of each feature map = its gradient averaged over height and width	alpha_k (the weights)	Each map gets one importance number. Averaging is the Grad-CAM rule
57	Weighted sum of the feature maps over channels	Linear combination	Adds up `alpha_k x A^k` into one 2 x 2 map
60	ReLU: keeps only positive values	ReLU	Keeps the regions that push the chosen score up, drops those pushing it down

```
        # Resize heatmap to original image dimensions
        cam = F.interpolate(
            cam,
            size=image.shape[2:],
            mode="bilinear",
            align_corners=False,
        )

        # Normalize to 0-1
        cam = cam[0, 0]
        cam = cam - cam.min()
        cam = cam / (cam.max() + 1e-8)

        return cam.detach().cpu(), predicted_class

    finally:
        forward_handle.remove()
        backward_handle.remove()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
63-68	Stretches the 2 x 2 map to the image size with bilinear interpolation	Upsampling	The heat map must match the 64 x 64 image to be overlaid. Nearest-neighbour would show blocks
71-73	Rescales to 0 to 1 (subtract min, divide by max)	Min-max normalisation	Makes colours comparable. The 1e-8 stops division by zero for an all-zero map. Note this always stretches the map so that its lowest point is 0
75	Returns the map and the predicted class, moved to the CPU	Return	Plotting libraries need CPU arrays

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
77-79	Removes both hooks, even if an error happened	`try / finally`	Without this, a second call would add a second hook and store wrong values

3.3 `src/gradcam_all_classes.py`: one correct and one wrong image per class

The script is run as `python -m src.gradcam_all_classes`. It uses rules to pick images, not personal choice.

```
"""Grad-CAM for one correct image of every class (first correct image in file order) plus one mistake per class.

Run from the repo folder:  python -m src.gradcam_all_classes
"""

import torch
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from PIL import Image
from torchvision import transforms
from pathlib import Path

from src.models import build_transfer_model
from src.gradcam import grad_cam
from src.utils import DATA_DIR, FIGURES_DIR, MODELS_DIR, RESULTS_DIR, SPLITS_CSV

device = torch.device("cpu")

class_names = [
    "AnnualCrop", "Forest", "HerbaceousVegetation",
    "Highway", "Industrial", "Pasture",
    "PermanentCrop", "Residential", "River", "SeaLake"
]

model = build_transfer_model("mobilenet_v2", freeze=False)
checkpoint = MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth"
model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
model.to(device)
model.eval()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Docstring: the rule for choosing images	Documentation	The rule is written down so nobody can say images were picked by hand
6-16	Imports	Libraries	Same helpers as the rest of the project
18	Runs on the CPU	Device	The model is small enough; the CPU is slower but works on any laptop
20-24	Class names in order 0 to 9	Labels	Must match the training labels
26-30	Builds the MobileNetV2 used in training, loads the final weights, sets evaluation mode	Transfer model, checkpoint	`freeze=False` means all layers were fine-tuned, as in the final model

```
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

splits = pd.read_csv(SPLITS_CSV)
test_df = splits[splits["split"] == "test"].reset_index(drop=True)
wrong_df = pd.read_csv(RESULTS_DIR / "misclassified_images.csv")

samples = []

for class_name in class_names:
```

```
    mistakes = wrong_df[wrong_df["actual_class"] == class_name]
    if len(mistakes) > 0:
        row = mistakes.iloc[0]
        image_path = DATA_DIR / row["path"]
        samples.append((
            class_name,
            Image.open(image_path).convert("RGB"),
            False
        ))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
32-39	Same preprocessing as training: resize to 64 x 64, to tensor, ImageNet normalisation	Preprocessing	The model only works well on inputs prepared the same way
41-43	Loads the split table and the saved mistakes list	CSV, filtering	Mistakes come from the file written by `evaluate.py`
45-56	For each class, takes the first wrong image in the mistakes list	Fixed rule	"First in the file" is a rule we can state. The mistakes list is in test-split order, so this is the first mistake in that order

```
correct_found = set()

for _, row in test_df.iterrows():
    class_name = row["class_name"]

    if class_name in correct_found:
        continue

    image_path = DATA_DIR / row["path"]
    image = Image.open(image_path).convert("RGB")

    input_tensor = transform(image).unsqueeze(0)

    with torch.no_grad():
        prediction = model(input_tensor).argmax(1).item()

    if prediction == row["label"]:
        samples.append((class_name, image, True))
        correct_found.add(class_name)

    if len(correct_found) == 10:
        break
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
58	Set of classes already given a correct example	Set	Stops after one correct image per class
60-66	Walks through the test rows in file order and opens each image	Loop	Same order every run
69-72	Predicts the image without storing gradients	`no_grad`	Only the prediction is needed here
74-76	Keeps the image if the prediction is right and the class is new	Fixed rule	"First correct image of each class in file order"
78-79	Stops once all 10 classes have one	Early stop	Avoids scanning the whole test set

Note: the file says "first correct image in file order". This is a rule, not a random sample, and it is a single image per class. It says nothing about how typical the image is.

```
fig, axes = plt.subplots(
    len(samples), 3,
    figsize=(12, len(samples) * 2.8)
)

if len(samples) == 1:
    axes = np.expand_dims(axes, axis=0)
```

```
for i, (class_name, image, is_correct) in enumerate(samples):
    input_tensor = transform(image).unsqueeze(0)

    heatmap, predicted_class = grad_cam(
        model,
        input_tensor,
        layer=model.features[-1]
    )

    image_np = np.array(image.resize((64, 64)))

    axes[i, 0].imshow(image_np)
    axes[i, 0].set_title(
        f"{class_name} - {'Correct' if is_correct else 'Wrong'}"
    )
    axes[i, 0].axis("off")

    axes[i, 1].imshow(heatmap, cmap="jet")
    axes[i, 1].set_title("Grad-CAM")
    axes[i, 1].axis("off")

    axes[i, 2].imshow(image_np)
    axes[i, 2].imshow(heatmap, cmap="jet", alpha=0.4)
    axes[i, 2].set_title(f"Predicted: {class_names[predicted_class]}")
    axes[i, 2].axis("off")

plt.tight_layout()

output = FIGURES_DIR / "gradcam_all_classes.png"
plt.savefig(output, dpi=150)
plt.close()

print("Grad-CAM saved:", output)
print("Images included:", len(samples))
print("Correct examples:", len(correct_found))
print("Misclassified examples:", len(samples) - len(correct_found))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
81-87	One row per image, three columns (picture, heat map, overlay)	matplotlib subplots	Shows the image, the heat map and the overlay side by side
89-96	Computes Grad-CAM for each chosen image on the last layer, `model.features[-1]`	Grad-CAM	The last block is the deepest that still has positions
98	Makes a 64 x 64 copy of the picture for display	Display	Keeps the picture and the heat map the same size
100-113	Draws the picture, the heat map in "jet" colours, and the heat map on top at 40% opacity	Overlay	The title shows the predicted class, so mistakes are visible
117-124	Saves the figure and prints counts (expected: 19 images, 10 correct, 9 wrong)	Output	The count check shows that SeaLake had no mistake, so only 9 wrong examples exist

3.4 `src/gradcam_confidence_drop.py`: hide the hottest 20% and measure the fall

The docstring says clearly that this test has no control. The control is `src/gradcam_sanity_check.py` (section 3.5).

```
"""Grad-CAM confidence-drop experiment.

Masks the hottest 20% of pixels of 50 correctly classified test images and measures how much the confidence falls.
Caution: there is NO control here (no coldest-area or random-area patch), so a small drop alone does not show that the
highlighted area matters. See src/gradcam_sanity_check.py for the control.

Run from the repo folder:  python -m src.gradcam_confidence_drop
"""

import torch
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt

from PIL import Image
from torchvision import transforms
from pathlib import Path

from src.models import build_transfer_model
from src.gradcam import grad_cam
from src.utils import DATA_DIR, FIGURES_DIR, MODELS_DIR, RESULTS_DIR

device = torch.device("cpu")
SAMPLE_SIZE = 50
MASK_PERCENT = 20
SEED = 42
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-8	Docstring: what is measured, the warning about no control	Documentation	The limitation is stated in the code itself
10-21	Imports	Libraries	Uses the same Grad-CAM function as the other scripts
24-26	Constants: 50 images, hide 20%, seed 42	Parameters	50 is small but enough for a first look; 20% is one choice, others were not tried
28-29	Sets the random seeds	Seed	Makes the shuffle repeatable

```
torch.manual_seed(SEED)
np.random.seed(SEED)

class_names = [
    "AnnualCrop", "Forest", "HerbaceousVegetation",
    "Highway", "Industrial", "Pasture",
    "PermanentCrop", "Residential", "River", "SeaLake"
]

model = build_transfer_model("mobilenet_v2", freeze=False)
checkpoint = MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth"
model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
model.to(device)
model.eval()

transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

df = pd.read_csv(DATA_DIR / "splits.csv")
df = df[df["split"] == "test"].sample(
    frac=1, random_state=SEED
).reset_index(drop=True)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
31-41	Class names and the trained model, in evaluation mode	Model	Same final weights as all of Part E
43-50	Same preprocessing as training	Preprocessing	Consistency
52-55	Reads the test rows and shuffles them with seed 42	Shuffle	The 50 images are a random sample from the test set, not the first 50

```
results = []

for _, row in df.iterrows():
    image_path = DATA_DIR / row["path"]
```

```
    original = Image.open(image_path).convert("RGB")
    image = transform(original).unsqueeze(0).to(device)

    # Original prediction and confidence
    with torch.no_grad():
        output = model(image)
        probabilities = torch.softmax(output, dim=1)
        predicted = output.argmax(dim=1).item()

    actual = int(row["label"])

    # Only use correctly classified images
    if predicted != actual:
        continue

    original_confidence = probabilities[0, actual].item()

    # Generate Grad-CAM
    heatmap, _ = grad_cam(model, image, model.features[-1])
    heatmap = heatmap.to(device)

    # Mask top 20% most important pixels
    threshold = torch.quantile(
        heatmap.flatten(), 1 - MASK_PERCENT / 100
    )
    mask = heatmap >= threshold

    masked_image = image.clone()
    masked_image[0, :, mask] = 0.0

    # Confidence after masking
    with torch.no_grad():
        masked_output = model(masked_image)
        masked_probabilities = torch.softmax(masked_output, dim=1)
        masked_confidence = masked_probabilities[0, actual].item()

    confidence_drop = original_confidence - masked_confidence

    results.append({
        "path": row["path"],
        "class_name": class_names[actual],
        "original_confidence": original_confidence,
        "masked_confidence": masked_confidence,
        "confidence_drop": confidence_drop
    })

    if len(results) >= SAMPLE_SIZE:
        break
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
59-62	Loads one test image and prepares it	Loop	Each image in shuffled order
65-68	Original probabilities and prediction	Softmax	Confidence is the probability of the chosen class
70-74	Skips images the model gets wrong	Filter	The test is about correct answers only
76	Stores the confidence of the true class	Confidence	Same class used later for the masked image
79-80	Grad-CAM for the image, moved to the device	Grad-CAM	The explanation for this image
83-86	Threshold at the 80th percentile; the mask is the pixels at or above it	`quantile`, mask	About 20% of pixels are selected. With `>=`, ties at the threshold can select a little more
88-89	Copies the image and sets the masked pixels to 0 in every channel	Masking	0 after normalisation is the ImageNet average colour, not the EuroSAT average. Also a solid colour can look unnatural to the model
92-95	Confidence of the true class after masking	Confidence	The second number of the pair

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
97	Drop = before minus after	Confidence drop	Positive means confidence fell. Negative means it rose (10 of 50 images)
99-105	Stores one row per image	Records	Feeds the CSV
107-108	Stops at 50	Limit	Sample size

```
results_df = pd.DataFrame(results)

output_csv = RESULTS_DIR / "gradcam_confidence_drop.csv"
results_df.to_csv(output_csv, index=False)

if len(results_df) > 0:
    mean_original = results_df["original_confidence"].mean()
    mean_masked = results_df["masked_confidence"].mean()

    plt.figure(figsize=(7, 5))
    plt.bar(
        ["Original", "Grad-CAM Masked"],
        [mean_original, mean_masked]
    )
    plt.ylabel("Average Confidence")
    plt.title("Confidence Drop After Grad-CAM Region Masking")
    plt.ylim(0, 1)
    plt.tight_layout()

    output_fig = FIGURES_DIR / "gradcam_confidence_drop.png"
    plt.savefig(output_fig, dpi=150, bbox_inches="tight")
    plt.close()

    print("Images tested:", len(results_df))
    print(f"Average original confidence: {mean_original:.4f}")
    print(f"Average masked confidence:  {mean_masked:.4f}")
    print(f"Average confidence drop:    {mean_original - mean_masked:.4f}")
    print("CSV saved:", output_csv)
    print("Graph saved:", output_fig)
else:
    print("No correctly classified images were processed.")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
112-113	Saves the 50 rows to `results/gradcam_confidence_drop.csv`	CSV	Lets anyone check the numbers
116-117	Averages original and masked confidence	Mean	The two numbers in the summary
119-131	Bar chart of the two means, saved as a figure	Plot	A picture only; the bars are close together, which is itself the message
133-138	Prints the counts and the average drop	Logging	Gives 0.0108 (1.08 points) on the saved run
139-140	Message if no image was processed	Defensive check	Avoids an empty-table error

3.5 `src/gradcam_sanity_check.py`: the control experiment

This file is the control for section 3.4. It was added by Nitin, not by Rohan. Its module docstring (not copied here) describes the same two checks as below. It runs on 200 test images: the first 20 of each class in `splits.csv` order.

```
import numpy as np
import pandas as pd
import torch
```

```
from src.dataset import EuroSATDataset
from src.gradcam import grad_cam
from src.train import build_model
from src.utils import MODELS_DIR, RESULTS_DIR

IMAGES_PER_CLASS = 20
HIDE = 1024                                  # 25% of the 64 x 64 = 4096 pixels
QUADRANTS = {(0, 0): "top-left", (0, 1): "top-right", (1, 0): "bottom-left", (1, 1): "bottom-right"}


def painted(batch, flat_ids):
    out = batch.clone().reshape(1, 3, -1)
    out[:, :, flat_ids] = 0.0                # 0 after normalisation = the average colour of the training data
    return out.reshape(1, 3, 64, 64)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
15-22	Imports	Libraries	Uses the project's dataset and model code
24	20 images per class	Sample size	200 images in total; a fixed rule, not chosen by look
25	HIDE = 1024 pixels = 25% of 64 x 64	Patch size	Same size for the hottest, coldest and random patch, so the comparison is fair
26	Maps the 2 x 2 quarter index to a name	Lookup	Used to report "bottom-right" and similar
29-32	Hides the chosen pixels in all three colour channels and returns the batch	Masking	Comment on line 31 says "the average colour of the training data". This is wrong: 0 after normalisation is the ImageNet mean colour (see section 2)

```
def main():
    model, _ = build_model("mobilenet_v2_full")
    state = torch.load(MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth", map_location="cpu", weights_only=True)
    model.load_state_dict(state)
    model.eval()

    data = EuroSATDataset("test")
    generator = torch.Generator().manual_seed(0)
    rows = []
    for label in range(10):
        for index in data.df.index[data.df["label"] == label][:IMAGES_PER_CLASS]:
            batch = data[int(index)][0].unsqueeze(0)
            heat, predicted = grad_cam(model, batch, model.features[-1])
            h = heat.numpy()
            quarters = np.array([[h[:32, :32].mean(), h[:32, 32:].mean()], [h[32:, :32].mean(), h[32:, 32:].mean()]])
            flat = heat.flatten()
            top, left = torch.randint(0, 33, (2,), generator=generator).tolist()
            square = ((torch.arange(32)[:, None] + top) * 64 + (torch.arange(32)[None, :] + left)).flatten()
            patches = {"hottest": flat.topk(HIDE).indices, "coldest": (-flat).topk(HIDE).indices, "random_square": square}
            with torch.no_grad():
                before = torch.softmax(model(batch), dim=1)[0, predicted].item()
                row = {"path": data.df.loc[index, "path"], "class_name": data.df.loc[index, "class_name"],
                       "hottest_quadrant": QUADRANTS[np.unravel_index(quarters.argmax(), (2, 2))]}
                for name, ids in patches.items():
                    probs = torch.softmax(model(painted(batch, ids)), dim=1)[0]
                    row[f"drop_{name}"] = before - probs[predicted].item()
                    row[f"changed_{name}"] = int(probs.argmax().item() != predicted)
            rows.append(row)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
36-39	Loads the final model in evaluation mode	Model	Same weights
41-43	Test set and a fixed random generator (seed 0)	Seed	The random squares are the same every run
45-46	Loops the first 20 images of each class, one at a time	Fixed rule	Reproducible sample

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
47	Grad-CAM for the image	Grad-CAM	Heat map used for both checks
48-52	Mean heat of each quarter of the map; picks a random 32 x 32 square	Quarter comparison; random patch	The quarter check is "where is the heat"; the random square is the third patch
53	Indices of the hottest and coldest 1,024 pixels, and the random square	`topk`, flattening	Three patches of equal area
55	Confidence of the chosen class before hiding	Confidence	Baseline
56-57	Row with the path, the class and the hottest quarter	Records	Feeds the CSV
58-61	For each patch: confidence after hiding, the drop, and whether the predicted class changed	Drop, prediction change	Two measures: how far confidence falls, and how often the answer flips
62	Adds the row	Records	One row per image

```
    table = pd.DataFrame(rows)
    table.to_csv(RESULTS_DIR / "gradcam_sanity_check.csv", index=False)

    n = len(table)
    print(f"Images tested: {n}")
    print("Check 1 - hottest quadrant of the Grad-CAM map:")
    for name in QUADRANTS.values():
        count = int((table["hottest_quadrant"] == name).sum())
        print(f"  {name:13s} {count:3d}  ({count / n:.0%})")
    print("Check 2 - hide 25% of the pixels:")
    for name in ("hottest", "coldest", "random_square"):
        print(f"  {name:14s} mean confidence drop {table[f'drop_{name}'].mean():.3f} | prediction changed for
{int(table[f'changed_{name}'].sum())} of {n}")
    hot, cold = table["drop_hottest"], table["drop_coldest"]
    print(f"  hottest hurt MORE than coldest for {int((hot > cold).sum())} images, LESS for {int((hot < cold).sum())}")


if __name__ == "__main__":
    main()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
64-65	Saves the table to `results/gradcam_sanity_check.csv`	CSV	200 rows
67-72	Counts the hottest quarter for each of the four quarters	Counting	Gives 160 bottom-right, 36 top-right, 4 bottom-left, 0 top-left
73-75	Mean drop and number of changed predictions for each patch	Summary	0.064 / 14, 0.061 / 14, 0.089 / 17
76-77	Counts images where hiding the hottest hurt more than the coldest	Paired comparison	109 more, 91 less
80-81	Runs `main()` only when the file is run directly	Entry point	Safe to import

3.6 `src/visualize_tsne.py`: features and t-SNE

The script has no functions; it runs top to bottom. It takes 1,000 random test images, extracts 1,280 numbers each, and plots a 2-D t-SNE.

```
"""visualize_tsne.py - MobileNetV2 feature visualization."""
import os

os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["MKL_NUM_THREADS"] = "1"
import csv
```

```
import random
import torch
import numpy as np
import matplotlib.pyplot as plt

from pathlib import Path
from PIL import Image
from torchvision import transforms
from sklearn.manifold import TSNE

from src.models import build_transfer_model
from src.utils import DATA_DIR, MODELS_DIR, RESULTS_DIR, FIGURES_DIR
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	One-line docstring	Documentation	
4-6	Limits the number of threads for the maths libraries to 1	Threads	Stops the maths libraries from fighting each other on a laptop and makes runs more stable. Set before the imports, so it takes effect
7-16	Imports	Libraries	t-SNE comes from scikit-learn
18-19	Model builder and project paths	Project code	Same model as the other scripts

```
# 1. Device
device = torch.device("cpu")


# 2. Load trained model
model = build_transfer_model("mobilenet_v2", freeze=False)

checkpoint = MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth"

model.load_state_dict(
    torch.load(checkpoint, map_location=device, weights_only=True)
)

model.to(device)
model.eval()


# 3. Image preprocessing
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
23	CPU device	Device	Enough for 1,000 images
27-36	Builds the model, loads the final weights, sets evaluation mode	Model	Same weights as Part E
40-47	Same preprocessing as training	Preprocessing	Consistency

```
# 4. Read test images using Python CSV reader
split_file = DATA_DIR / "splits.csv"

test_rows = []

with open(split_file, "r", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)

    for row in reader:
```

```
        if row["split"] == "test":
            test_rows.append(row)

print("Total test images:", len(test_rows))


# Randomly select 1000 test images (fixed seed 42)
random.Random(42).shuffle(test_rows)
test_rows = test_rows[:1000]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
51	Path of the split table	Path	
55-60	Reads the rows with Python's csv module and keeps the test rows	CSV reading	Avoids pandas here; same result
62	Prints the number of test rows (4,050)	Logging	
66-67	Shuffles with seed 42 and keeps 1,000	Random sample, seed	Fixed sample. The plan asked for about 2,000 images; the code uses 1,000 (see section 8)

```
# 5. Extract features
features_list = []
labels_list = []

print("Extracting MobileNet features...")

with torch.no_grad():

    for i, row in enumerate(test_rows):

        image_path = DATA_DIR / row["path"]

        image = Image.open(image_path).convert("RGB")
        image_tensor = transform(image).unsqueeze(0).to(device)

        feature_maps = model.features(image_tensor)

        features = torch.nn.functional.adaptive_avg_pool2d(
            feature_maps, (1, 1)
        )

        features = features.flatten(1)

        features_list.append(features.cpu().numpy()[0])
        labels_list.append(int(row["label"]))

        if (i + 1) % 250 == 0:
         print(f"Processed {i + 1}/1000 images")


features = np.array(features_list)
labels = np.array(labels_list)

print("Feature shape:", features.shape)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
76	No gradients	`no_grad`	Only features are needed
85	Runs only the feature part of MobileNetV2 (`model.features`)	Features before the classifier	This is the output of the last convolution block, before dropout and the new 10-class layer. Gives a feature map per image
87-91	Averages each map to one number (global average pooling) and flattens	Global average pooling	1,280 numbers per image. Could use the 1,000-class ImageNet output instead, which is less specific to our task
93-94	Stores the feature vector and the label	Records	

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
96-97	Progress message every 250 images	Logging	Shows that a long run is working
100-103	Makes arrays and prints the shape (1000, 1280)	NumPy	t-SNE needs an array

```
# 6. Run t-SNE
print("Running t-SNE. This may take several minutes on CPU...")

tsne = TSNE(
    n_components=2,
    perplexity=30,
    random_state=42,
    init="pca",
    learning_rate="auto",
)

embeddings = tsne.fit_transform(features)

print("t-SNE completed!")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
109-115	t-SNE: 2 dimensions, perplexity 30, seed 42, PCA start, automatic learning rate	t-SNE	Perplexity 30 is the usual default and the plan asked for it. The PCA start and seed make the layout repeatable on the same library version. Could use PCA first to reduce 1,280 to 50 dimensions, which is faster
117	Runs t-SNE and gets 2-D points	`fit_transform`	Takes minutes on a CPU

```
# 7. Plot
class_names = [
    "AnnualCrop", "Forest", "HerbaceousVegetation",
    "Highway", "Industrial", "Pasture",
    "PermanentCrop", "Residential", "River", "SeaLake"
]

plt.figure(figsize=(12, 9))

for class_id, class_name in enumerate(class_names):

    mask = labels == class_id

    plt.scatter(
        embeddings[mask, 0],
        embeddings[mask, 1],
        label=class_name,
        s=8,
        alpha=0.7,
    )

plt.title("t-SNE Visualization of MobileNetV2 Features")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.legend(markerscale=2, fontsize=8)
plt.tight_layout()


# 8. Save
FIGURES_DIR.mkdir(parents=True, exist_ok=True)

output_path = FIGURES_DIR / "mobilenet_v2_tsne.png"

plt.savefig(output_path, dpi=120)
plt.show()

print("t-SNE figure saved:", output_path)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
123-127	Class names	Labels	Colours and legend
129-141	One scatter plot per class, small dots, slight transparency	Scatter plot	Overlapping classes show as mixed colours
143-147	Title, axis labels, legend	Plot	The axis numbers have no unit; only their relative positions mean something
151-155	Saves the figure before showing it	Save order	Saving after `plt.show()` can write an empty image in some setups; this order is correct
156	Opens a window; close it to finish	Interactive display	Only for the person running it
158	Prints the save path	Logging	

3.7 `src/train_part_e.py`: seed-4 scratch CNN runs (brief)

This is Rohan's extra seed (seed 4) for the scratch CNN, run on the CPU, as the plan says. It re-implements a short training loop with the same settings as the team: batch 64, Adam, learning rate 0.001, up to 20 epochs, best validation epoch kept, the test set scored once.

```
"""Memory-efficient Part E scratch CNN seed-4 experiments."""

import time
import numpy as np
import pandas as pd
import torch
from sklearn.metrics import f1_score

from src.dataset import get_loader
from src.models import build_scratch_cnn
from src.train import _save_row
from src.utils import get_device, set_seed, METRICS_CSV

BATCH_SIZE = 64
EPOCHS = 20
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
2	Docstring	Documentation	
4-13	Imports, including `_save_row` from `train.py`	Libraries, reuse	Same CSV writing rule as the shared function
15-16	Batch 64, 20 epochs	Settings	Same as the team recipe

```
def train_part_e(fraction, seed=4):
    device = get_device()
    set_seed(seed)

    train_loader = get_loader(
        "train", fraction=fraction, seed=seed,
        augment=True, batch_size=BATCH_SIZE
    )
    val_loader = get_loader("val", batch_size=BATCH_SIZE)
    test_loader = get_loader("test", batch_size=BATCH_SIZE)

    model = build_scratch_cnn().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, foreach=False)
    loss_fn = torch.nn.CrossEntropyLoss()

    best_accuracy = -1
    best_weights = None
```

```
    best_epoch = 0
    started = time.perf_counter()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
19	Function takes the fraction (0.05, 0.1, 0.25) and seed 4	Parameters	
21	Sets all seeds	Seed	Repeatable run
23-28	Training loader with augmentation and fraction; validation and test loaders	DataLoader	Augmentation only on training images
30-32	Scratch CNN, Adam, cross-entropy	Model, optimiser, loss	Adam with `foreach=False` is the setting Tejash used, so results match
34-37	Best accuracy, best weights, best epoch, start time	Model selection	Decisions use the validation set only

```
    for epoch in range(1, EPOCHS + 1):
        model.train()
        total_loss, correct, seen = 0.0, 0, 0

        for x, y in train_loader:
            x, y = x.to(device), y.to(device)

            optimizer.zero_grad()
            scores = model(x)
            loss = loss_fn(scores, y)
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * len(y)
            correct += (scores.argmax(1) == y).sum().item()
            seen += len(y)

        model.eval()
        val_correct, val_seen = 0, 0
        val_loss = 0.0

        with torch.no_grad():
            for x, y in val_loader:
                x, y = x.to(device), y.to(device)
                scores = model(x)
                val_loss += loss_fn(scores, y).item() * len(y)
                val_correct += (scores.argmax(1) == y).sum().item()
                val_seen += len(y)

        val_accuracy = val_correct / val_seen

        print(
            f"scratch_cnn | fraction {fraction:g} | seed {seed} | "
            f"epoch {epoch:02d}/{EPOCHS} | "
            f"train loss {total_loss/seen:.4f}, "
            f"acc {correct/seen:.3f} | "
            f"val loss {val_loss/val_seen:.4f}, "
            f"acc {val_accuracy:.3f}"
        )

        if val_accuracy > best_accuracy:
            best_accuracy = val_accuracy
            best_epoch = epoch
            best_weights = {
                k: v.detach().cpu().clone()
                for k, v in model.state_dict().items()
            }
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
40-54	One pass over the training images: forward, loss, backward, step	Training loop	The standard recipe

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
56-66	Validation pass without gradients	Validation	Chooses the epoch
68-77	Prints loss and accuracy for each epoch	Logging	Lets us see training curves in the output
79-85	Keeps a copy of the best weights	Early selection	Copy is on the CPU so memory is not used on the GPU

Note: the code's `__main__` block (lines 122-123) runs only the 25% case. The 5% and 10% rows in `metrics.csv` were produced by calling the same function with other fractions; no saved command for them is in the file.

```
    model.load_state_dict(best_weights)
    model.eval()

    actual, predicted = [], []

    with torch.no_grad():
        for x, y in test_loader:
            scores = model(x.to(device))
            predicted.extend(scores.argmax(1).cpu().tolist())
            actual.extend(y.tolist())

    accuracy = np.mean(np.array(actual) == np.array(predicted))
    macro_f1 = f1_score(actual, predicted, average="macro")

    row = {
        "model": "scratch_cnn",
        "fraction": fraction,
        "seed": seed,
        "accuracy": accuracy,
        "macro_f1": macro_f1,
    }

    _save_row(METRICS_CSV, row)

    seconds = time.perf_counter() - started

    print(
        f"\nTEST (best epoch {best_epoch}): "
        f"accuracy {accuracy:.4f}, macro-F1 {macro_f1:.4f} "
        f"| {seconds/60:.1f} min"
    )

    return row
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
87-88	Restores the best weights and switches to evaluation	Restore	The test score is for the best epoch, not the last
92-96	Scores the test set once	Test set	Never used to choose anything
98-99	Accuracy and macro-F1	Metrics	Same definitions as `evaluate.py`
101-109	Builds the row and writes it to `metrics.csv`	CSV, one row per run	A rerun replaces its old row (rule in `train.py`)
111-117	Prints the test result and the time	Logging	

```
if __name__ == "__main__":
    train_part_e(fraction=0.25, seed=4)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
122-123	Runs the 25% case with seed 4 when the file is run	Entry point	See the note above about 5% and 10%

3.8 `src/visualize_confusion_examples.py`: real examples of confused pairs

```
"""Real examples (first three in the file) of the six most confused class pairs.

Run from the repo folder:  python -m src.visualize_confusion_examples
"""

import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

from src.utils import DATA_DIR, FIGURES_DIR, RESULTS_DIR

df = pd.read_csv(RESULTS_DIR / "misclassified_images.csv")

pairs = [
    ("AnnualCrop", "PermanentCrop"),
    ("AnnualCrop", "Pasture"),
    ("HerbaceousVegetation", "Pasture"),
    ("HerbaceousVegetation", "PermanentCrop"),
    ("Highway", "River"),
    ("Pasture", "AnnualCrop"),
]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Docstring: first three examples of the six pairs	Documentation	
6-10	Imports	Libraries	
12	Loads the mistakes list	CSV	
14-21	The six pairs: AnnualCrop->PermanentCrop, AnnualCrop->Pasture, HerbaceousVegetation->Pasture, HerbaceousVegetation->PermanentCrop, Highway->River, Pasture->AnnualCrop	Fixed list	These are the top six pairs by count (7, 5, 5, 5, 4, 4). The list is typed in by hand, so if the model changed, it would need updating

```
fig, axes = plt.subplots(6, 3, figsize=(12, 20))

for row, (actual, predicted) in enumerate(pairs):
    examples = df[
        (df["actual_class"] == actual) &
        (df["predicted_class"] == predicted)
    ].head(3)

    for col, (_, item) in enumerate(examples.iterrows()):
        image_path = DATA_DIR / item["path"]
        image = Image.open(image_path).convert("RGB")

        axes[row, col].imshow(image)
        axes[row, col].set_title(
            f"Actual: {actual}\nPredicted: {predicted}",
            fontsize=9
        )
        axes[row, col].axis("off")

    for col in range(len(examples), 3):
        axes[row, col].axis("off")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
23	Grid of 6 rows and 3 columns	Subplots	One row per pair

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
25-29	Picks the first three mistakes of that pair in file order	Filter, `head`	Fixed rule, not chosen by eye
31-40	Shows each image with the real and the predicted class in its title	Plot	
42-43	Blank panel when a pair has fewer than three examples	Layout	Keeps the grid tidy

```
plt.suptitle(
    "Examples of Most Confused EuroSAT Class Pairs",
    fontsize=16
)
plt.tight_layout()

output = FIGURES_DIR / "confusion_pair_examples.png"
output.parent.mkdir(parents=True, exist_ok=True)
plt.savefig(output, dpi=200, bbox_inches="tight")
plt.close()

print(f"Saved visualization: {output}")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
45-49	Title and layout	Plot	
51-56	Saves at 200 dpi and prints the path	Save	

3.9 `src/visualize_errors.py`: grid of mistakes

```
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image

from src.utils import DATA_DIR, RESULTS_DIR

# Load the incorrect predictions
df = pd.read_csv(RESULTS_DIR / "misclassified_images.csv")

# Select 12 incorrect images
df = df.head(12).reset_index(drop=True)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
2-6	Imports	Libraries	
9	Loads the mistakes list	CSV	
12	Keeps the first 12 mistakes in file order	Fixed rule	Not the worst 12, not chosen by eye

```
# Create a 3-row, 4-column grid
fig, axes = plt.subplots(3, 4, figsize=(16, 12))
axes = axes.flatten()

for i, row in df.iterrows():

    # Find the image
    image_path = DATA_DIR / row["path"]

    # Open image
    img = Image.open(image_path).convert("RGB")

    # Display image
    axes[i].imshow(img)

    # Show actual and predicted classes
    axes[i].set_title(
        f"Actual: {row['actual_class']}\n"
```

```
        f"Predicted: {row['predicted_class']}",
        fontsize=9,
        pad=8
    )

    axes[i].axis("off")

# Hide unused plots
for i in range(len(df), len(axes)):
    axes[i].axis("off")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
15-16	A 3 x 4 grid, flattened to a list of 12 panels	Subplots	
18-27	Opens each image and shows it	Plot	
30-37	Title with the real and predicted class	Labels	
40-41	Turns off unused panels	Layout	Here all 12 are used

```
# Improve spacing
plt.subplots_adjust(
    hspace=0.35,
    wspace=0.15,
    top=0.95,
    bottom=0.03
)

# Save the figure
output_path = RESULTS_DIR / "misclassified_images_grid.png"

plt.savefig(
    output_path,
    dpi=120,
    bbox_inches="tight"
)

plt.show()

print("Image grid saved:", output_path)
print("Images displayed:", len(df))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
44-49	Spacing between panels	Layout	
52-62	Saves to `results/misclassified_images_grid.png` (note: results folder, not figures) and shows it	Save, display	The notebook reads it from `results/`

3.10 `src/visualize_gradcam.py`: Grad-CAM for the first four mistakes

This script is an older version of the picture in `gradcam_all_classes.py`. It shows only the first four mistakes, in two columns.

```
import torch
import matplotlib.pyplot as plt
import pandas as pd
from PIL import Image
from torchvision import transforms

from src.models import build_transfer_model
from src.gradcam import grad_cam
from src.utils import DATA_DIR, MODELS_DIR, RESULTS_DIR

# Device
device = torch.device("cpu")

# Load trained MobileNetV2
model = build_transfer_model("mobilenet_v2", freeze=False)
```

```
checkpoint = MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth"
model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
model.to(device)
model.eval()

# Image preprocessing
transform = transforms.Compose([
    transforms.Resize((64, 64)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-9	Imports; uses the same Grad-CAM function	Reuse	One Grad-CAM for the whole project
11-20	CPU; final MobileNetV2 weights; evaluation mode	Model	
22-30	Same preprocessing as training	Preprocessing	

```
# Read misclassified images
df = pd.read_csv(RESULTS_DIR / "misclassified_images.csv")

# Select first 4 mistakes
df = df.head(4).reset_index(drop=True)

class_names = [
    "AnnualCrop", "Forest", "HerbaceousVegetation",
    "Highway", "Industrial", "Pasture",
    "PermanentCrop", "Residential", "River", "SeaLake"
]

fig, axes = plt.subplots(4, 2, figsize=(10, 16))

for i, row in df.iterrows():

    image_path = DATA_DIR / row["path"]
    original = Image.open(image_path).convert("RGB")

    image_tensor = transform(original).unsqueeze(0).to(device)

    # Generate Grad-CAM
    heatmap, predicted_class = grad_cam(
        model,
        image_tensor,
        model.features[-1]
    )
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
33-36	Loads the mistakes list and keeps the first four	Fixed rule	
38-42	Class names	Labels	Defined here but not used later (harmless)
44	A 4 x 2 grid	Subplots	Picture and heat map
46-58	Grad-CAM for each of the four images	Grad-CAM	Last layer `model.features[-1]`

```
    heatmap = heatmap.numpy()

    # Original image
    axes[i, 0].imshow(original)
    axes[i, 0].set_title(
        f"Actual: {row['actual_class']}\n"
        f"Predicted: {row['predicted_class']}"
    )
    axes[i, 0].axis("off")

    # Heatmap overlay
```

```
    axes[i, 1].imshow(original)
    axes[i, 1].imshow(heatmap, cmap="jet", alpha=0.5)
    axes[i, 1].set_title("Grad-CAM")
    axes[i, 1].axis("off")

plt.tight_layout()

output_path = RESULTS_DIR / "figures" / "gradcam_examples.png"
plt.savefig(output_path, dpi=120, bbox_inches="tight")
plt.show()

print("Grad-CAM figure saved:", output_path)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
63-68	Shows the picture with real and predicted class	Plot	
71-74	Shows the heat map on top of the picture at 50% opacity	Overlay	
76-82	Saves to `results/figures/gradcam_examples.png` and shows it	Save	The figure is shown in notebook block 5

3.11 `notebooks/04_evaluation_and_explainability.ipynb`: code blocks

The notebook runs from top to bottom and does not retrain anything. Sections 1 to 8 show what the scripts saved. Section 9 (added by Nitin) re-scores the model on the CPU to check the saved files. The code blocks are numbered 1 to 15 in the order they appear. Line numbers count lines inside the block.

Block 1: load the saved report and matrix (section 1).

```
from pathlib import Path
import os
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Image

# Works whether Jupyter starts in the repo folder or in notebooks/.
project_root = Path.cwd()
if not (project_root / "src").is_dir():
    project_root = project_root.parent
os.chdir(project_root)
RESULTS = project_root / "results"
FIGURES = RESULTS / "figures"

report = pd.read_csv(
    RESULTS / "mobilenet_v2_full_frac1_seed0_classification_report.csv",
    index_col=0
)
cm = pd.read_csv(
    RESULTS / "mobilenet_v2_full_frac1_seed0_confusion_matrix.csv",
    index_col=0
)

print("Classification Report")
display(report)

print("Confusion Matrix")
display(cm)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-5	Imports	Libraries	`display` shows tables nicely in Jupyter
7-13	Finds the project folder, whether the notebook opens from the project or from `notebooks/`, and sets the paths	Paths	Lets the notebook run from either folder

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
15-22	Reads the report and the matrix from the CSV files, with the class names as the index	CSV	Reads what the scripts saved, so nothing is retrained
24-28	Prints both tables	Display	The numbers used in the write-up

Block 2: draw the confusion matrix (section 2).

```
plt.figure(figsize=(10, 8))
plt.imshow(cm.values, cmap="Blues")
plt.colorbar()
plt.xticks(range(len(cm.columns)), cm.columns, rotation=45, ha="right")
plt.yticks(range(len(cm.index)), cm.index)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("MobileNetV2 Confusion Matrix")

for i in range(len(cm.index)):
    for j in range(len(cm.columns)):
        plt.text(j, i, cm.iloc[i, j], ha="center", va="center")

plt.tight_layout()
plt.show()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Sets up a 10 x 8 figure with a blue colour map	Plot	Same colours as the script
5-8	Tick labels and axis titles	Plot	
10-12	Writes each count inside its cell	Annotation	
14-15	Layout and show	Plot	

Block 3: show the mistakes grid (section 3).

```
display(Image(filename=str(RESULTS / "misclassified_images_grid.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Shows the saved PNG from `visualize_errors.py`	Display	Shows the picture, not a new run

Block 4: show the pair examples (section 3.1).

```
display(Image(filename=str(FIGURES / "confusion_pair_examples.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Shows the saved pair figure	Display	

Block 5: show the Grad-CAM examples (section 4).

```
display(Image(filename=str(FIGURES / "gradcam_examples.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Shows the saved 2-column Grad-CAM figure	Display	

Block 6: show Grad-CAM for all classes (section 4.1).

```
display(Image(filename=str(FIGURES / "gradcam_all_classes.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Shows the saved all-classes figure	Display	

Block 7: confidence-drop table and picture (section 5).

```
confidence = pd.read_csv(RESULTS / "gradcam_confidence_drop.csv")
display(confidence.head())
print("Number of evaluated images:", len(confidence))
display(Image(filename=str(FIGURES / "gradcam_confidence_drop.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-2	Reads the 50-row CSV and shows the first five	Pandas	The first five show that one drop is negative
3	Prints 50	Count check	
4	Shows the bar chart	Display	

Block 8: control summary (section 5.1).

```
check = pd.read_csv(RESULTS / 'gradcam_sanity_check.csv')
summary = pd.DataFrame({
    'mean confidence drop': [check[f'drop_{n}'].mean() for n in ('hottest', 'coldest', 'random_square')],
    'prediction changed (of %d)' % len(check): [int(check[f'changed_{n}'].sum()) for n in ('hottest', 'coldest', 'random_square')],
}, index=['hottest 25%', 'coldest 25%', 'random 32 x 32 square']).round(3)
display(summary)
print('Hottest quarter of the heat map:')
print(check['hottest_quadrant'].value_counts().to_string())
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Reads the 200-row control CSV	CSV	
2-7	Builds a small table: mean drop and changed predictions for the three patches	DataFrame	Same numbers as section 3.5
8	Shows the table	Display	
9-11	Prints the count of hottest quarters	Counting	160, 36, 4

Block 9: t-SNE picture (section 6).

```
display(Image(filename=str(FIGURES / "mobilenet_v2_tsne.png")))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Shows the saved t-SNE figure	Display	

Block 10: seed-4 rows (section 7).

```
metrics = pd.read_csv(RESULTS / "metrics.csv")
display(metrics[metrics["seed"] == 4])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-2	Reads `metrics.csv` and shows the rows with seed 4	Filter	Three rows: 5%, 10%, 25%. Note these are the stored values; the notebook does not run anything

Block 11: re-score the model on the CPU (section 9).

```
import sys

import numpy as np
import torch
from PIL import Image
```

```
sys.path.insert(0, str(project_root))
from src.dataset import EuroSATDataset
from src.train import build_model
from src.utils import DATA_DIR, MODELS_DIR

RUN = 'mobilenet_v2_full_frac1_seed0'
CLASSES = ['AnnualCrop', 'Forest', 'HerbaceousVegetation', 'Highway', 'Industrial',
           'Pasture', 'PermanentCrop', 'Residential', 'River', 'SeaLake']

model, _ = build_model('mobilenet_v2_full')
model.load_state_dict(torch.load(MODELS_DIR / f'{RUN}.pth', map_location='cpu', weights_only=True))   # weights_only: safe loading
model.eval()

test_set = EuroSATDataset('test')
test_df = test_set.df
labels = test_df['label'].to_numpy()

probabilities = []
with torch.no_grad():
    for start in range(0, len(test_set), 256):
        batch = torch.stack([test_set[i][0] for i in range(start, min(start + 256, len(test_set)))])
        probabilities.append(torch.softmax(model(batch), dim=1))
probabilities = torch.cat(probabilities).numpy()
predictions = probabilities.argmax(axis=1)
top_prob = probabilities.max(axis=1)

mine = pd.crosstab(pd.Series(labels, name='actual'), pd.Series(predictions, name='predicted')).reindex(
    index=range(10), columns=range(10), fill_value=0).to_numpy()
print(f'Correct on this run: {int((predictions == labels).sum())} of {len(labels)}  (Rohan saved {int(np.trace(cm.values))})')
print(f'Cells that differ from his saved matrix (sum of absolute differences): {int(np.abs(mine - cm.values).sum())}')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-10	Imports; adds the project folder to the import path; imports the dataset, model and paths	Imports	
12-14	The run name and the class names	Labels	
16-18	Builds the model, loads the final weights, evaluation mode	Model	`weights_only=True` for safe loading
20-22	Test set, its table and labels	Test set	
24-31	Loops in batches of 256 to get the softmax probabilities; takes the argmax and the top probability	Softmax, confidence	The probability of the chosen class is the confidence
33-34	Builds a 10 x 10 table from the new predictions	Confusion matrix	Built the same way as the saved one
35-36	Compares: 3,981 correct, and a difference of 0 cells from the saved matrix	Reproduction check	Shows the saved matrix is reproduced exactly on this run

Block 12: per-class table sorted by recall (section 9.1).

```
per_class = report.loc[CLASSES, ['precision', 'recall', 'f1-score', 'support']].copy()
per_class['support'] = per_class['support'].astype(int)
per_class.sort_values('recall').round(3)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Keeps the four columns for the 10 classes in the class order	Selection	
2	Turns support into whole numbers	Types	
3	Sorts by recall, rounds to 3 decimals	Sorting	Sorting by recall puts PermanentCrop first, which is the weakest. Note the sort is written in the block but the display shows the result

Block 13: most confused pairs (section 9.2).

```
pairs = sorted(((int(cm.loc[a, p]), a, p) for a in CLASSES for p in CLASSES if a != p and cm.loc[a, p] > 0),
               key=lambda t: (-t[0], t[1], t[2]))
mistakes_total = sum(t[0] for t in pairs)
print(f'{len(pairs)} different (actual -> predicted) pairs happened, {mistakes_total} mistakes in total.\n')
for count, actual, predicted in pairs[:8]:
    print(f'  {count} x {actual:22s} -> {predicted}')

crops = {'AnnualCrop', 'PermanentCrop', 'Pasture', 'HerbaceousVegetation'}
inside = sum(t[0] for t in pairs if t[1] in crops and t[2] in crops)
print(f'\nThe 6 biggest pairs are {sum(t[0] for t in pairs[:6])} of {mistakes_total} mistakes.')
print(f'Both the real and the predicted class are crop or vegetation classes: {inside} of {mistakes_total} ({inside /
mistakes_total:.0%}).')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-2	Lists every (count, actual, predicted) where the count is above zero, sorted by count	Sorting	Gives 29 pairs
3	Total mistakes: 69	Sum	
4-6	Prints the top eight pairs	Output	
8-9	Counts mistakes where both classes are crop or vegetation classes	Set membership	36 of 69
10-11	Prints the top six total (30 of 69) and the crop share (52%)	Output	

Block 14: real examples with confidence (section 9.3).

```
mistakes = pd.read_csv(RESULTS / 'misclassified_images.csv')
row_of_path = {p: i for i, p in enumerate(test_df['path'])}
top_pairs = pairs[:6]

fig, axes = plt.subplots(len(top_pairs), 4, figsize=(10, 2.7 * len(top_pairs)))
for r, (count, actual, predicted) in enumerate(top_pairs):
    chosen = mistakes[(mistakes['actual_class'] == actual) & (mistakes['predicted_class'] == predicted)].head(4)
    for c in range(4):
        ax = axes[r, c]
        ax.axis('off')
        if c < len(chosen):
            row = chosen.iloc[c]
            ax.imshow(Image.open(DATA_DIR / row['path']).convert('RGB'))
            ax.set_title(f"{top_prob[row_of_path[row['path']]]:.0%} sure", fontsize=9)
    axes[r, 0].text(-0.08, 0.5, f'{count} x {actual}\n-> {predicted}', transform=axes[r, 0].transAxes, ha='right', va='center',
fontsize=9)
fig.suptitle('Most confused pairs: real mistakes (first four of each pair)', y=0.995)
fig.tight_layout()
fig.savefig(FIGURES / 'most_confused_pairs.png', dpi=110, bbox_inches='tight')
plt.show()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-3	Mistake list, row number of each path, the top six pairs	Lookup	
5-8	A grid: six rows of four panels	Subplots	
9-11	Takes the first four mistakes of each pair and leaves the rest of the row blank	Fixed rule	"First four in file order"
12-14	Shows each image with its confidence in the title	Confidence	Shows how sure the model was of its wrong answer
15	Writes the pair name on the left	Labels	
16-19	Title, layout, saves `most_confused_pairs.png`, shows it	Save	The figure is the one in `results/figures/`

Block 15: is the model unsure when it is wrong? (section 9.4).

```
wrong = predictions != labels
print(f'Mean confidence when right: {top_prob[~wrong].mean():.3f}   (median {np.median(top_prob[~wrong]):.3f})')
print(f'Mean confidence when wrong: {top_prob[wrong].mean():.3f}   (median {np.median(top_prob[wrong]):.3f})')
for limit in (0.6, 0.8, 0.9):
    print(f'Mistakes below {limit:.0%} confidence: {int((top_prob[wrong] < limit).sum())} of {int(wrong.sum())}'
          f'   | correct answers below {limit:.0%}: {int((top_prob[~wrong] < limit).sum())} of {int((~wrong).sum())}')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Finds the wrong predictions	Boolean	
2-3	Mean and median confidence, right and wrong	Summary	0.990 (median 1.000) when right; 0.763 (median 0.772) when wrong
4-6	For limits 60%, 80%, 90%: how many mistakes and how many correct answers fall below	Threshold analysis	At 80%: 39 of 69 mistakes, but also 60 of 3,981 correct answers

4. How Grad-CAM works, step by step

Grad-CAM answers: which places in the last feature map made the chosen class score go up? It needs one forward pass, one backward pass and two hooks.

1. **Forward pass.** The tile goes through MobileNetV2. The forward hook on `model.features[-1]` keeps the activations `A^k` (1,280 maps of 2 x 2 for our 64 x 64 input, because the network divides the size by 32).
2. **Score.** Take the score of the chosen class (the logit, before softmax), `y^c`.
3. **Backward pass.** `score.backward()` gives the gradient `dy^c / dA^k` at every position of every map. The backward hook keeps these.
4. **Weights (alpha).** For each map k, average the gradient over the positions:

```
alpha_k^c = (1 / Z) x sum over i and j of (dy^c / dA^k_ij)
```

where Z = number of positions = 2 x 2 = 4. In the code this is `gradients.mean(dim=(2, 3))`. A large alpha means: making this map stronger would raise the class score a lot. 5. **Weighted sum and ReLU.** Combine the maps and keep only positive values:

```
L^c = ReLU( sum over k of alpha_k^c x A^k )
```

The ReLU keeps the regions that support the class. Negative regions are dropped. 6. **Resize.** Bilinear interpolation stretches the 2 x 2 map to 64 x 64. 7. **Normalise.** Subtract the minimum and divide by the maximum, so the map runs from 0 to 1. 8. **Overlay.** Draw the map in "jet" colours on the image, semi-transparent.

Why the last convolution layer? It is the deepest layer that still has spatial positions. Deeper layers know more about the class, but after global average pooling the position is gone. Earlier layers have finer grids (for a 64 x 64 input, the earlier blocks have 16 x 16, 8 x 8 and 4 x 4 positions) but respond to simpler things like edges. The last block is the best trade-off for "what" and "where", but on 64 x 64 tiles it only has 2 x 2 positions, which is why our maps are coarse.

What Grad-CAM is not. It is one method, one layer, one image. It does not prove what the model "thinks". Our control (section 6) found the heat maps do not single out a more important area on this model.

5. The mistakes: which classes are confused and why

Overall. 69 of 4,050 test tiles are wrong (1.7%). The test set has 450 images of most classes and 300 of Pasture, so accuracy and macro-F1 are close here (98.30% and 98.20%).

Per class (from the classification report, the CPU re-scoring of section 9.1 gives the same numbers):

Class	Precision	Recall	F1	Support
PermanentCrop	0.968	0.960	0.964	375
AnnualCrop	0.982	0.971	0.977	450
HerbaceousVegetation	0.987	0.976	0.981	450
Pasture	0.951	0.977	0.964	300
Highway	0.973	0.979	0.976	375
Industrial	0.989	0.981	0.985	375
River	0.989	0.987	0.988	375
Residential	0.989	0.996	0.992	450
Forest	0.993	0.998	0.996	450
SeaLake	0.996	1.000	0.998	450

PermanentCrop has the lowest recall (0.960: 15 of 375 missed). Pasture has the lowest precision (0.951): other green classes are called Pasture more often than Pasture is missed. SeaLake, Forest and Residential are almost perfect.

Where the 69 mistakes go. The 29 different (actual -> predicted) pairs that happened are led by these:

Mistakes	Actual	Predicted as
7	AnnualCrop	PermanentCrop
5	AnnualCrop	Pasture
5	HerbaceousVegetation	Pasture
5	HerbaceousVegetation	PermanentCrop
4	Highway	River
4	Pasture	AnnualCrop

- **The crop and vegetation group.** 36 of the 69 mistakes (52%) are between AnnualCrop, PermanentCrop, Pasture and HerbaceousVegetation (checked against the matrix: 12 from AnnualCrop, 10 from HerbaceousVegetation, 5 from Pasture, 9 from PermanentCrop). Four classes that look alike from above make half of all mistakes.
- **AnnualCrop against PermanentCrop.** The largest single pair, 7 images. The brief expected this pair, and our data confirms it, but it is not the only one.
- **Highway against River.** Highway -> River 4 and River -> Highway 3, so 7 mistakes in total between these two. The plan expected this pair too; our data shows it as a real but smaller confusion, not the biggest one.
- **Pasture.** Involved in three of the six largest pairs.

What the examples show. The figure `results/figures/most_confused_pairs.png` shows the first four real mistakes of each of the six largest pairs, with the model's confidence in its wrong answer. What I can see in it:

- AnnualCrop called PermanentCrop: bright, dry-looking ploughed fields with field boundaries and tracks. The model is only 61 to 79% sure. These are the hardest and least confident examples.
- AnnualCrop called Pasture: dark green, uniform fields with faint boundaries. Several are 64 to 78% sure, one is 100% sure.
- HerbaceousVegetation called Pasture or PermanentCrop: mixed, patchy vegetation, some bare ground. Confidence ranges from 54% to 99%.
- Highway called River: here the road sits next to water or a river, and both are in the tile, so the label is partly a matter of which feature is dominant. Confidence 50 to 92%.
- Pasture called AnnualCrop: green fields with thin lines between them. 53 to 88% sure.

Possible reasons, which we did not test: at 64 x 64 pixels (about 10 m per pixel) the difference between a ploughed field and a grass field is mostly fine texture; some tiles really are ambiguous; and the random split can put neighbouring patches of one field in both training and test. These are guesses from looking at pictures, and the notebook says so.

Confidence. The model is 99.0% sure on average when it is right and 76.3% sure when it is wrong (CPU re-scoring, section 9.4). A rule "send to a person when under 80% sure" would catch 39 of 69 mistakes but also flag 60 of 3,981 correct answers. The threshold was looked at on the test set, so it is an illustration, not a tuned rule. The main lesson is that mistakes are not always unsure: 56 of 69 were below 90%, but the model still gives a high probability to its wrong answer in many cases.

6. What the Grad-CAM control showed

The first test (confidence drop). For 50 correctly classified test images we hid the hottest 20% of pixels (set to the ImageNet average colour) and measured the confidence of the true class.

- Average confidence before: 97.72%. After: 96.64%. Average drop: 1.08 points (0.0108).
- In 10 of the 50 images the model became more confident after hiding the "hot" area.
- A second laptop re-run gave 0.61 points on the same 50 images (as recorded in `results/part_e_analysis.md`; that value is not in the saved CSV).

On its own this looks like "the hot area matters a little". It does not show that, because nothing was compared with it.

The control (`gradcam_sanity_check.py`). For 200 test images, three patches of 25% of the pixels were hidden in turn: the hottest 25%, the coldest 25%, and a random 32 x 32 square.

Patch hidden	Mean confidence drop	Prediction changed
Hottest 25% (Grad-CAM)	0.064	14 of 200
Coldest 25% (Grad-CAM)	0.061	14 of 200
Random 32 x 32 square	0.089	17 of 200

- Hiding the hottest area hurt more for 109 images and less for 91.
- Hottest minus coldest, paired on the same 200 images: mean difference 0.003, standard error 0.017, p = 0.87 (my own paired t-test on the saved CSV; not in the repo). This is no difference at all.
- The random square did slightly more damage than the hottest patch (paired p = 0.08, not significant at 5%).
- The medians are very small (hottest 0.0008, coldest 0.0002, random 0.0014): most patches barely change the confidence. The means are driven by a few images with big drops.

Where the heat is. The hottest quarter of the 2 x 2 map was the bottom-right in 160 of 200 images (80%), the top-right in 36, the bottom-left in 4 and the top-left in none. The same lower-right pattern can be seen in `gradcam_all_classes.png`: the heat is in the lower-right for many images whatever the content.

What this shows. On this model and at this size, the Grad-CAM heat map does not identify a part of the tile that matters more than the rest. Hiding the hottest area does no more harm than hiding the coldest, so the heat maps are not evidence of what the model uses. It also shows the maps mostly show a position (bottom-right) rather than the content of the image.

What this does NOT prove.

- It does not prove Grad-CAM is useless in general. It is one method, on one model, one layer, one image size.
- It does not prove the model ignores the image content. Our guess that the model uses texture spread over the tile is not tested.

- The patch is a solid colour, which is unnatural and can itself change the model's output in ways unrelated to importance. The first version of the control used scattered random pixels; it was dropped because scattered noise damages an image far more than one solid patch, so that comparison was unfair.
- The confidence is very close to 1 for most correct images (mean 0.98), so there is little room for a drop. A ceiling effect can hide a real difference.
- The top-k "hottest" pixels form an irregular blob, while the random patch is a square. The shapes differ, so the comparison is not perfectly fair either.
- 200 images, one seed, one model.

Ideas not tried (also in the notebook): Grad-CAM on an earlier block with a finer grid, or enlarging the tile before Grad-CAM, and a control with other fill colours.

7. What t-SNE shows and what it does not show

The plot `results/figures/mobilenet_v2_tsne.png` uses 1,000 random test images, each as the 1,280 numbers after global average pooling, reduced to 2 dimensions with perplexity 30.

What it shows.

- Ten groups, one per class. Each group is a compact region. The features of unseen test images are organised by class even though t-SNE was never told the labels. It only uses the features; the colours are added afterwards.
- PermanentCrop (pink) sits in the middle, between HerbaceousVegetation (green, above) and AnnualCrop (blue, below), with a few points at the edge. This is the picture of the 7 + 5 mistakes in the confusion matrix.
- Pasture (brown) is a separate group next to HerbaceousVegetation and PermanentCrop.
- Highway (red) and River (yellow) are separate groups, far apart from each other.
- Residential (grey) and Industrial (purple) are close on the left, with a few grey points at the edge of the purple group. Matches the Industrial -> Residential and Residential -> Industrial mistakes.
- SeaLake (cyan) is separate, with a small sub-group at the bottom.

What it does not show.

- It does not show how far apart two groups really are. Distances between groups on a t-SNE plot have no meaning, and neither do the sizes of the groups.
- It does not show accuracy. Points inside a group can still be wrong, and a mistake can sit in the middle of the right group.
- It does not show the model's decision boundary. The 10-class layer that decides is not in the plot.
- The picture depends on the settings. Another perplexity or seed would give a different layout. We used one seed (42) and one perplexity (30).
- Our sample is 1,000 images, not the 2,000 in the plan. The plan's number was not used in the code, the notebook or the report (see section 8).
- The model was trained on these classes, so clean groups are expected. The useful point is that unseen test images still group well.

8. Numbers you must know

Item	Value	Where it comes from
Test images	4,050	`classification_report.csv` (support total)
Correct / wrong	3,981 / 69	`misclassified_images.csv`, confusion matrix
Accuracy (CPU re-scoring, log and analysis)	98.30%	`part_e_analysis.md`, notebook
Accuracy (GPU run, the official number in `metrics.csv`)	98.27%	`metrics.csv`, seed 0. The GPU run gives 3,980 right, the CPU run 3,981: one image apart

Item	Value	Where it comes from
Macro-F1 (CPU report)	98.20% (0.9820)	classification report
Macro-F1 (`metrics.csv`)	98.18%	`metrics.csv`
PermanentCrop recall (lowest)	0.960 (360 of 375)	classification report
Pasture precision (lowest)	0.951	classification report
Mistakes between the four crop/vegetation classes	36 of 69 (52%)	confusion matrix
Biggest pair	AnnualCrop -> PermanentCrop, 7	confusion matrix
Highway -> River / River -> Highway	4 / 3	confusion matrix
Mean confidence right / wrong	0.990 / 0.763	notebook section 9.4
Confidence-drop test	50 images, 97.72% -> 96.64%, drop 1.08 points	`gradcam_confidence_drop.csv`
Confidence-drop re-run on second laptop	0.61 points	`part_e_analysis.md` only (not saved in CSV)
Control: hottest / coldest / random drop	0.064 / 0.061 / 0.089	`gradcam_sanity_check.csv` (200 images)
Control: prediction changed	14 / 14 / 17 of 200	same
Control: hottest vs coldest	109 more, 91 less	same
Hottest quarter of heat map	bottom-right 160 (80%), top-right 36, bottom-left 4, top-left 0	same
Grad-CAM map size	2 x 2 cells, stretched to 64 x 64	MobileNetV2 last block, stride 32
t-SNE sample	1,000 test images, seed 42, perplexity 30	`visualize_tsne.py`
t-SNE feature size	1,280	`model.features` plus global average pooling
Seed 4, scratch CNN, 5%	74.69% accuracy, 73.96% macro-F1	`metrics.csv`
Seed 4, scratch CNN, 10%	81.68% / 80.92%	`metrics.csv`
Seed 4, scratch CNN, 25%	87.43% / 86.81%	`metrics.csv`
Seed 4, re-run on second laptop, 5% (CPU)	74.44%	`part_e_analysis.md` only
Seed 4, shared GPU training function, 5 / 10 / 25%	78.62 / 81.46 / 85.31%	`part_e_analysis.md` only (not in `metrics.csv`)

Things I must say honestly in the viva.

- The plan asked for about 2,000 t-SNE images; the code and all the write-ups use 1,000.
- The plan asked for Grad-CAM on ResNet-18 as well; all Grad-CAM files use MobileNetV2 only.
- Two numbers for the same model: 98.30% (CPU) and 98.27% (GPU). Say which one is which.
- The control was written by Nitin, not by Rohan.
- The comment in `gradcam_sanity_check.py` about "average colour of the training data" is wrong; it is the ImageNet average.
- `train_part_e.py` runs only the 25% case in its main block.
- The Highway -> River example is a visual guess, not a tested reason.